DeepSeek V4.1 Flash decode: plan to raise tok/s/user

Blackhole Galaxy, 32 chips · batch 1, 32k context · Nov 2, 2026

Today: 27.9 ms per token, **36 tok/s/user** (measured). **Plan:** move the MoE to 12 dedicated chips at TP12 and keep attention at TP4 on the other 20. Use the L1 that frees up to preload attention weights, and fuse the attention block into one kernel. **Expected:** ~20 ms per token, **~51 tok/s/user** (~1.4x). Range: ~22 ms (45 tok/s/user) if every lever lands at its worse end, ~18 ms (55 tok/s/user) at its best.

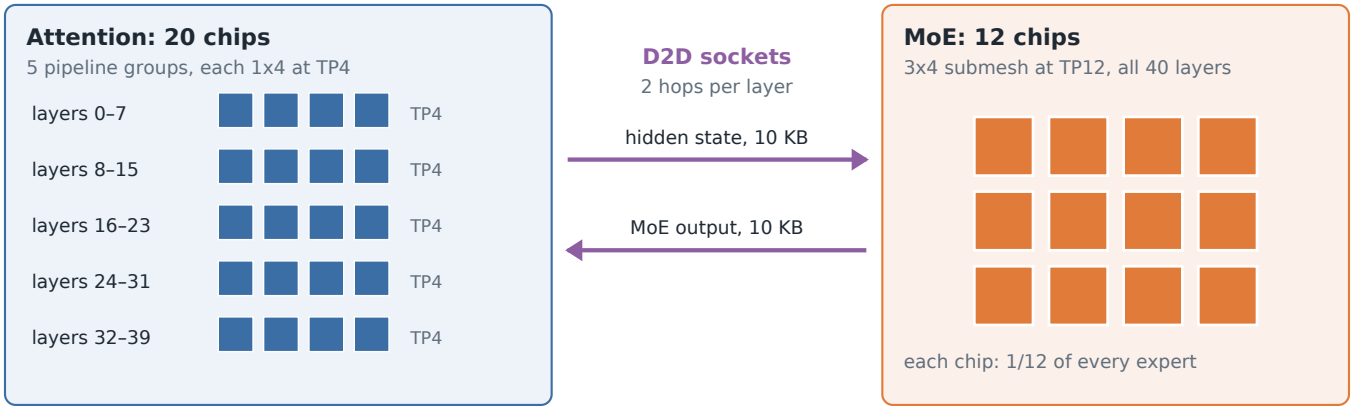
1. Why decode is slow today

Today's layout is 8 pipeline stages of 4 chips (TP4), 5 layers per stage. Every chip runs both attention and MoE for its layers, so the two compete for the same L1 and DRAM. At batch 1, 28 of the 32 chips are idle while one stage works. One layer takes ~530 µs:

Part	Per layer	Per token	Why it is slow
Attention (TP4)	~188 µs	7.5 ms	Each chip reads 21.7 MB of weights per layer from DRAM while the matmuls run. The prefetcher is only 2–4 pages deep, so it streams alongside the matmul instead of ahead of it. A consumer layer is 14 separate ops, with 66–85 µs of fixed cost: output RoPE on one core (23 µs), the o_b all-reduce (15–34), RoPE + cache write (10.5), SDPA (12.9).
MoE (TP4)	193 µs	7.7 ms	Each chip reads 1/4 of each of the 6 chosen experts (~30 MB). `fused_experts` 165 µs, router 15, small ops 9, all-reduce 5. Its 540 KB-per-core weight buffer also sets the L1 high-water mark, which leaves no room to keep attention weights resident.
Other per-layer work (mHC, norms)	~150 µs	~6.0 ms	Replicated on all 4 chips. Outside this plan.
Stage hops, trace boundaries, host		~6.7 ms	The rest of the 27.9 ms. Outside this plan.
Token		**27.9 ms**	**36 tok/s/user**

Sources: layer and token times from the Oct 27 / Oct 29 production runs (traced, prefetcher on). Attention is that layer minus MoE and other work; the per-op detail is from the Nov 2 eager TP4 profile, which reads 238 µs per layer. MoE from the Nov 2 single-layer profile (`MOE_TP12_SUMMARY.md`).

2. How to improve



MoE: TP12 on its own 12 chips

- **TP, not EP.** Every chip reads 1/12 of each chosen expert, so one token gets 3x the DRAM bandwidth it gets at TP4. Expert parallelism would put the 6 experts on at most 6 chips and give no per-user gain.
- **Done and measured on one layer:** 193 → ~112 µs (6 unpadded I-tiles per chip, shared expert moved to an idle core, `gate_up` spread over all 8 DRAM banks).
- **D2D sockets** (`MeshSocket`, as in the d2d_exchange micro-op of deepseek_v3_b1) carry the 10 KB hidden state to the MoE chips and the output back: 2 hops per layer.

- **Next, toward < 50 µs:** remove the ~28 µs of setup before the first weight read, fix the slow hub cores in `gate_up` (25–37 µs against a 6 µs median), and shard the router matmul across the 12 chips.

Attention: TP4, deeper prefetch, fused kernels

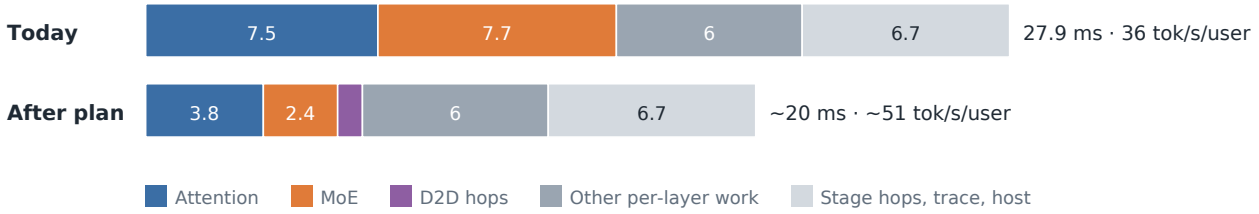
- **Keep TP4.** Measured 2x faster than one chip (238 vs 468 µs per layer).
- **Deeper prefetch (whole-layer L1 preload).** With the MoE gone, use the freed L1 to hold the next layer's attention weights: 21.7 MB per chip, ~180 KB per core over 120 cores. The attention chips' DRAM is idle while the MoE chips run and during the ~150 µs of mHC and norm work, and loading one layer takes only ~54 µs at 400 GB/s. The matmuls then read L1, which takes DRAM latency off the critical path.
- **Fused attention-block kernels.** Replace the 14 ops with one program: q/kv RoPE and cache write in the matmul epilogues (10.5 → 3–8 µs), output RoPE on 16 cores or in the SDPA reducer (23 → 3–6), all-reduce fused into o_b (15–34 → 8–15), more SDPA cores per head (12.9 → 8–11), all-gathers folded in (4.6 → 2–4). Fixed cost goes from 66–85 µs to 24–45 µs, and the launch and sync gaps between ops go away.
- **Evaluate a Python-level fusion API.** deepseek_v3_b1 already builds its fused attention block in Python: micro-ops (rmsnorm, matmul, rope, kv_cache_update, flash_mla, all-reduce) are composed into one `ttnn.generic_op` program (`fused_ops/attention_block`, `unified_kernel_descriptor.py`). Try it on one V4.1 chain (q_b → RoPE → cache write) against a hand-written op. Compare speed, the effort to add a new micro-op, and how it works with trace and the prefetcher. Decide before writing the full block.

3. Potential perf gains and why

Expected case: each lever lands mid-range. No credit for work outside this plan.

Step (cumulative)	Per token	tok/s/user	Why
Today: 8 stages x TP4	27.9 ms	36	Measured.
1. MoE on 12 chips at TP12, D2D sockets	~25.5 ms	~39	MoE 193 → 112 µs per layer saves 3.2 ms per token. Socket hops at ~10 µs each (one all-reduce hop measures ~5 µs), so the 80 hops add 0.8 ms.
2. MoE kernel fixes	~23.4 ms	~43	Most of the 28 µs setup goes (−20 µs), the hub cores come close to the median (−20), and the router is sharded (−12). MoE ~60 µs per layer saves 2.1 ms.
3. Attention: L1 preload + fused kernels	~19.7 ms	~51	Attention 7.5 → 3.8 ms. Consumer layers (32 of 40) ~58 µs: matmuls from L1 ~25 µs, fused fixed-cost ops ~33 µs. The 8 index layers ~240 µs: their compressor and indexer matmuls gain from the preload too, but the select work stays.

Not counted: 4 stage hops instead of 7, fewer op launch gaps after fusion, and the indexer plan (attention to ~3.0 ms). Per-token time, ms (after plan: expected case):



Risks and what to measure first

- D2D hop latency is not measured yet. Every 10 µs per hop is 0.8 ms per token, so measure it first.
- The attention baseline (7.5 ms) and the per-op data (9.5 ms, eager, no prefetcher) come from different runs. Re-profile one layer traced with the prefetcher on to get both from the same run.
- The L1 preload needs each matmul's weights on the cores that consume them, in that matmul's shard layout. The 180 KB-per-core figure assumes an even spread, and it shares L1 with Engram and the indexer's whole-grid select. Index layers also carry bf16 compressor weights.
- Each MoE chip holds 25.5 GB of routed experts (of 32 GB): no room for bf8 experts or duplicated hot experts.
- At high concurrency the 12 MoE chips do every expert read with 37.5% of today's expert DRAM bandwidth, so aggregate throughput can drop (`DECODE_PARALLELISM_SUMMARY.md` section 5).
- After this plan, ~12.7 of the ~20 ms is other per-layer work and stage/host gaps. That is the next target.